SQL Project Documentation

Name: Amol Baviskar

Project Overview & Objective:

In this project, I analyzed a fictional music database using SQL to answer business questions and uncover insights related to customers, sales, and music genres. This was an opportunity for me to apply my SQL skills, including joins, grouping, filtering, sorting, and using CTEs and window functions for advanced queries.

Table Structure:

Assume the database has the following tables:

1. **employee**: Contains details of employees, including their job title and seniority levels.
2. **invoice**: Contains all invoices with billing information.
3. **customer**: Contains customer data, such as first and last names and contact information.
4. **invoice_line**: Detailed information on each item within an invoice.
5. **track**: Details of each track, including genre and duration.
6. **album**: Contains information on music albums.
7. **artist**: Information on artists.
8. **genre**: Information on genres.

Each table has relevant fields like customer_id, invoice_id, track_id, artist_id, etc., for joining tables.

Implementation:

Creating Database and Tables: Used DDL commands to create Database and Tables including the relation by using primary and foreign key constraints.

Ingestion of data from dataset: Used MYSQL Workbench 8.0 to import data from excel file into respective tables.

Questions based on the table data:

1. Easy Level Queries:

• Q1: Find the most senior employee based on job title.

o Hint: Use the employee table and sort by the levels column in descending order.

→

```
mysql> SELECT employee_id, first_name, last_name, title, levels FROM employee ORDER BY levels DESC LIMIT 1;
+-------------+------------+-----------+-----------------+--------+
| employee_id | first_name | last_name | title           | levels |
+-------------+------------+-----------+-----------------+--------+
|           1 | Andrew     | Adams     | General Manager | L6     |
+-------------+------------+-----------+-----------------+--------+
1 row in set (0.12 sec)
```

Q2: Determine which countries have the most invoices.

o Hint: Group the invoice data by billing_country, then count and sort the results.

→

```
mysql> select billing_country, count(*) as total_invoices
    -> from invoice
    -> group by billing_country
    -> order by total_invoices desc;
```

billing_country	total_invoices
USA	131
Canada	76
Brazil	61
France	50
Germany	41
Czech Republic	30
Portugal	29
United Kingdom	28
India	21
Ireland	13
Chile	13
Finland	11
Spain	11
Poland	10
Denmark	10
Australia	10
Hungary	10
Sweden	10
Netherlands	10
Norway	9
Italy	9
Austria	9
Belgium	7
Argentina	5

• Q3: Identify the top 3 invoice totals.
o Hint: Sort the invoice table by the total column.

```
mysql> select invoice_id, total from invoice
    -> order by total desc
    -> limit 3;
+------------+-------+
| invoice_id | total |
+------------+-------+
|        183 | 23.76 |
|         92 | 19.80 |
|         31 | 19.80 |
+------------+-------+
3 rows in set (0.00 sec)
```

• Q4: Find the city with the highest total invoice amount to determine the best location for a promotional event.

→

```
mysql> select sum(total) as invoice_total , billing_city from invoice
    -> group by billing_city
    -> order by invoice_total desc
    -> limit 1;
+---------------+--------------+
| invoice_total | billing_city |
+---------------+--------------+
|        273.24 | Prague       |
+---------------+--------------+
1 row in set (0.04 sec)
```

• Q5: Identify the customer who has spent the most money.
o Hint: Use a join between customer and invoice, group by customer_id, and sum the totals.

→

```
mysql> select c.customer_id, c.first_name, c.last_name, SUM(i.total) as total_spent
    -> from customer c
    -> join invoice i
    -> on c.customer_id = i.customer_id
    -> group by c.customer_id, c.first_name, c.last_name
    -> order by total_spent desc
    -> limit 1;
+-------------+------------+--------------+-------------+
| customer_id | first_name | last_name    | total_spent |
+-------------+------------+--------------+-------------+
|           5 | FrantiÅ¡ek | WichterlovÃ¡ |      144.54 |
+-------------+------------+--------------+-------------+
1 row in set (0.01 sec)
```

2. Moderate Level Queries:

• Q1: Find the email, first name, and last name of customers who listen to Rock music.

o Hint: Use joins across customer, invoice, invoice_line, and track, filtering for the genre Rock.

→

```
mysql> select distinct c.first_name, c. last_name, c.email, g.name
    -> from customer c join invoice i on c.customer_id = i.customer_id
    -> join invoice_line il on i.invoice_id=il.invoice_id
    -> join track t on il.track_id=t.track_id
    -> join genre g on t.genre_id=g.genre_id
    -> where g.name = 'Rock'
    -> order by c.email asc;
```

Q2: Identify the top 10 rock artists based on track count.

o Hint: Use joins across artist, album, and track and filter by the genre Rock. Count tracks per artist.

→

```
mysql> select ar.artist_id, ar.name AS artist_name,
    -> count(t.track_id) as total_tracks
    -> from artist ar join album al on ar.artist_id = al.artist_id
    -> join track t on al.album_id = t.album_id
    -> join genre g on t.genre_id = g.genre_id
    -> where g.name = 'Rock'
    -> group by ar.artist_id, ar.name
    -> order by total_tracks desc
    -> limit 10;
+-----------+--------------------------------+--------------+
| artist_id | artist_name                    | total_tracks |
+-----------+--------------------------------+--------------+
|         1 | AC/DC                          |           18 |
|         3 | Aerosmith                      |           15 |
|         8 | Audioslave                     |           14 |
|        22 | Led Zeppelin                   |           14 |
|         4 | Alanis Morissette              |           13 |
|         5 | Alice In Chains                |           12 |
|        23 | Frank Zappa & Captain Beefheart |            9 |
|         2 | Accept                         |            4 |
+-----------+--------------------------------+--------------+
8 rows in set (0.09 sec)
```

Q3: Find all track names that are longer than the average track length.

o Hint: Calculate the average length and compare each track's length to this average.

→

```
mysql> select name,milliseconds
    -> from track
    -> where milliseconds > (select avg(milliseconds) from track)
    -> order by milliseconds desc;
```

3. Advanced Level Queries:

Q1: Calculate how much each customer has spent on each artist.

o Hint: Use a CTE to calculate the earnings per artist from invoice_line, then join it
with customer, invoice, and artist.

→

```
mysql> WITH best_selling_artist AS (
    ->     SELECT
    ->         ar.artist_id,
    ->         ar.name AS artist_name,
    ->         SUM(il.unit_price * il.quantity) AS total_sales
    ->     FROM invoice_line il
    ->     JOIN track t ON il.track_id = t.track_id
    ->     JOIN album al ON t.album_id = al.album_id
    ->     JOIN artist ar ON al.artist_id = ar.artist_id
    ->     GROUP BY ar.artist_id, ar.name
    ->     ORDER BY total_sales DESC
    ->     LIMIT 1
    -> )
    -> SELECT
    ->     c.customer_id,
    ->     c.first_name,
    ->     c.last_name,
    ->     bsa.artist_name,
    ->     SUM(il.unit_price * il.quantity) AS amount_spent
    -> FROM customer c
    -> JOIN invoice i
    ->     ON c.customer_id = i.customer_id
    -> JOIN invoice_line il
    ->     ON i.invoice_id = il.invoice_id
    -> JOIN track t
    ->     ON il.track_id = t.track_id
    -> JOIN album al
    ->     ON t.album_id = al.album_id
    -> JOIN best_selling_artist bsa
    ->     ON al.artist_id = bsa.artist_id
    -> GROUP BY
    ->     c.customer_id,
    ->     c.first_name,
    ->     c.last_name,
    ->     bsa.artist_name
    -> ORDER BY amount_spent DESC;
```

Q2: Determine the most popular music genre for each country based on purchases.

o Hint: Use a CTE or window function to rank genres by purchase count per country.

→

```
mysql> with popular_genre as (
    -> select c.country, g.name as genre_name, g.genre_id,
    -> count(il.invoice_line_id) AS total_purchases,
    -> dense_rank() over (
    -> partition by c.country
    -> order by count(il.invoice_line_id) desc) as rank_no
    -> from customer c join invoice i on c.customer_id = i.customer_id
    -> join invoice_line il on i.invoice_id = il.invoice_id
    -> join track t on il.track_id = t.track_id
    -> join genre g on t.genre_id = g.genre_id
    -> group by c.country,g.name,g.genre_id)
    -> select country, genre_name, total_purchases
    -> from popular_genre
    -> where rank_no = 1
    -> order by country asc;
```

Q3: Identify the top-spending customer for each country.

o Hint: Calculate the total spending per customer per country and filter for the highest spending.

→

```
mysql> with customer_spending as (
    -> select c.country, c.customer_id, c.first_name, c.last_name,
    -> sum(i.total) as total_spent, dense_rank() over (partition by c.country
    -> order by sum(i.total) desc) as rank_no
    -> from customer c join invoice i
    -> on c.customer_id = i.customer_id
    -> group by c.country, c.customer_id, c.first_name, c.last_name)
    -> select country,customer_id, first_name,last_name, total_spent
    -> from customer_spending
    -> where rank_no = 1
    -> order by country asc;
```